

Assignment: Neural Network Classification of Brain Tumor Type

You are given a tabular medical dataset of 9,000 patient cases. Each record has 27 features, a mix of numeric and categorical variables together with a target label, `tumor_type`, which takes one of three values: Glioma, Meningioma, or Pituitary. The data is imperfect: it contains missing values recorded under several inconsistent markers, categorical fields requiring encoding, features on very different scales, class imbalance, and some features are uninformative.

Your task is to design, train, tune, and critically evaluate a feed-forward neural network (multilayer perceptron) that classifies a case into one of the three tumour types. You are responsible for the entire workflow, from cleaning and preprocessing the raw dataset to selecting and justifying one final model.

Submit your final trained model so it can be tested on a separate held out dataset by the examiner.

Files provided

File	Contents
<code>brain_tumor_data.csv</code>	Dataset: 9,000 records, 27 features + target
<code>data_dictionary.txt</code>	Column descriptions, value ranges, missing-value markers

Rules and constraints

- Fit all preprocessing transformations (imputation values, scalers, encoders) on training data only, then apply them unchanged to validation and test partitions. Fitting any transformation on data you later evaluate on is data leakage and will not be correct.
- Set and report a random seed so your results are reproducible.
- All code must be your own and clearly commented. If you use AI assistants, you must declare how, where, and to what extent (see Section – Academic Integrity).

Tasks

Part A - Data understanding and preprocessing

Step 1 - Load and inspect.

- Load the dataset and report its shape, column data types, and the first few rows.
- Identify which columns are numeric and which are categorical. Which column is an identifier that should be excluded from modelling, and why?

Step 2 - Missing values.

- Convert all markers to a single consistent representation (NaN). Show the resulting count of missing values per column.
- Some numeric columns load as text because of a string marker. Identify them and explain why, then correct their data type.
- Choose and justify an imputation strategy for the numeric and the categorical columns respectively. Explain why the imputer must be fitted on training data only.

Step 3 - Encoding and scaling.

- Encode the categorical features. Justify your choice between one-hot and ordinal encoding for named columns (consider which are truly ordinal).
- Encode the target labels into a form suitable for a 3-class neural network. State whether you use integer labels or one-hot, and how this constrains your output layer and loss function.
- Scale the numeric features. Explain why feature scaling matters for gradient-based training and which scaler you chose.

Step 4 - Class balance.

- Report the class distribution. Is it balanced?
- State one consequence of this imbalance for (a) training and (b) evaluation, and name one technique you could use to mitigate it.

Part B - Model design

Step 1 - Architecture.

- Design a feed-forward neural network for this problem. Specify the number of input units, hidden layers, units per layer, and output units, and justify each choice with reference to the data.
- Choose activation functions for the hidden layers and the output layer. Why is the output activation determined by the number of classes?
- State your loss function and explain why it is the correct choice for multi-class classification given your label encoding from Part A Step 3.

Step 2 - Regularisation.

- Add at least one regularisation mechanism (ex. dropout, L2 weight decay, batch normalisation, early stopping). Explain what overfitting is and how your chosen mechanism reduces it.
- Provide a diagram or printed summary of your final architecture and report the total number of trainable parameters.

Part C - Data splitting, training, and tuning

Step 1 - Design your evaluation protocol.

- Partition the dataset into training, validation, and test subsets. State the proportions, whether you stratified by class, and justify the scheme you chose. Explain the distinct role each subset plays.
- Explain why the test subset must be set aside before any tuning and used only once, at the very end.

Step 2 - Training setup.

- Choose an optimiser, learning rate, and batch size. Justify the starting values.
- Train the model and plot training vs validation loss and accuracy across epochs.
- Interpret the curves: is the model underfitting, overfitting, or well-fit? Cite specific evidence from the plots.

Step 3 - Hyperparameter tuning.

- Vary **all** hyperparameters (e.g. number of layers, units, learning rate, dropout rate, batch size). Present results as a comparison.
- Select your single best model and justify the choice using validation performance only, not test performance.

Part D - Final evaluation

Apply the preprocessing pipeline fitted on your training data, unchanged, to your held-out test subset. Do not re-fit anything and do not revisit tuning after seeing these results.

Step 1 - Core metrics.

- Report overall accuracy on your test subset.
- Report precision, recall, and F1-score **per class**, plus macro- and weighted-averaged F1. Explain the difference between macro and weighted averaging and which is more informative here.

Step 2 - Confusion matrix.

- Produce and display the confusion matrix.
- Which two classes are most often confused with each other? Propose a plausible reason based on the data dictionary and feature distributions.

Step 3 - Confidence.

- Pick two cases the model classifies with high confidence and two with low confidence. Show the predicted class probabilities and comment on what distinguishes them.

Part E - Critical analysis and reflection

Step 1 - Baseline comparison.

- Train one non-neural baseline (e.g. logistic regression or random forest) on the same splits. Compare its test performance with your neural network in a table.
- Was the neural network worth the added complexity for this dataset? Justify with evidence.

Step 2 - Feature insight.

- Identify which features your model relies on most (ex. via permutation importance, or by ablating features and re-measuring performance). Name two features that appear largely uninformative.

Step 3 - Limitations and ethics.

- State two limitations of your model or your evaluation.
- Discuss one risk of deploying a model like this in a real clinical setting, and one safeguard you would require before deployment.

Deliverables

1. Notebook (.ipynb) : runnable end to end from a fixed seed, reproducing all results.
2. Report (PDF, max 12 pages) : answers to Parts A–E with the requested tables and plots.
3. Final model file: your trained model saved to .h5 file, plus the fitted preprocessing pipeline and a short predict.py that loads raw data in the same format as brain_tumor_data.csv, applies your preprocessing, and outputs predicted classes. Your model will be run on a separate held-out dataset in this exact format, so it must run without manual editing.
4. Your preprocessing for new data must be reproducible from the saved artifacts, do not hard-code values from your training run by hand.

Academic integrity

State clearly whether and how you used generative AI tools. Permitted uses (ex: explaining an error message, suggesting plotting code) must be declared in an appendix with the prompts used. Submitting AI-generated analysis or written reflection as your own is an academic offence and will be penalized.

Submission

- Submit all required components in a one Zip File via e-Kel by 30th of June 2026.
- Name files studentID_NN.ipynb, studentID_NN.pdf, and studentID_model.h5 and the Zip file studentID_all.zip.